



微處理機基本概念

Fundamentals of Microprocessors

從過去到未來，
一顆晶片改變了世界！



機械計算

電機機械

真空管

電晶體

積體電路

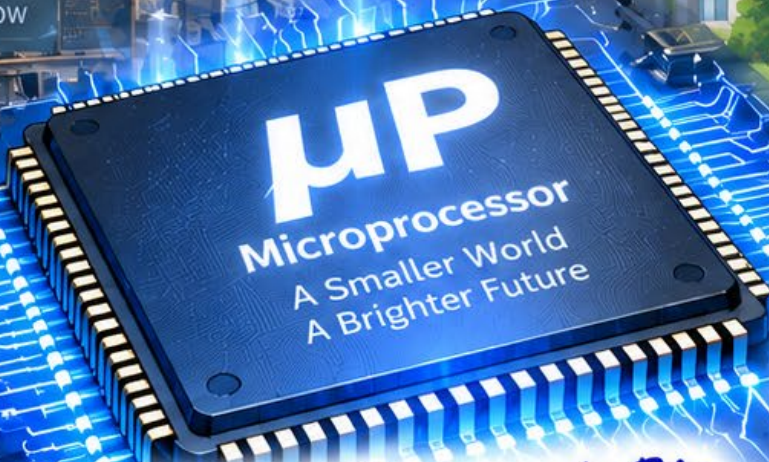
微處理機

現代應用

一起探索
微處理機，
發現改變世界
的力量！

拓真 Takuma
熱愛科技 · 勇於挑戰

From
Calculators
to a Smarter
Tomorrow



Think Small Think Big

微處理機
就在我們身邊，
讓生活更便利，
世界更美好！

玲奈 Reina
喜歡思考 · 樂於學習

Technology
People
A Better Tomorrow

1-2

這一章，我們要學什麼？

從過去到現在，帶你掌握微處理機的關鍵知識！

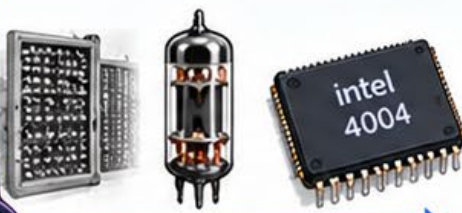
跟著我們，
一步一步
建立扎實的
基礎！

拓真 Takuma

熱愛科技・勇於挑戰

1 微處理機怎麼來的？

從早期的機械計算、真空管電腦到半導體技術的突破，了解微處理機的發展歷程。



機械計算 真空管 半導體 微處理機

2 微處理機裡面有什麼？

認識微處理機的基本組成單元，包括控制單元、算術/邏輯運算單元、暫存器與匯流排，了解各單元的功能與彼此的關係。



3 CPU 怎麼執行指令？

了解指令的提取、解碼與執行流程，以及如何透過機器語言來控制整個系統的運作。



看起來複雜，
其實很有邏輯



微處理機
就在我們生活中，
一起來發現
它的有趣與重要性
吧！

玲奈 Reina

喜歡思考・樂於學習

學完這一章，你將能夠...



掌握

微處理機的發展脈絡



認識

微處理機的內部結構



理解

CPU 的指令執行流程



建立

後續章節學習的基礎

打好基礎，
才能走得更遠！
下一頁，我們從
人類如何計算開始！



Think
Design
Build
a Better
Tomorrow

1-3

沒有微處理機以前，人類怎麼計算？

How Did People Compute Before the Microprocessor?

從手動計算到自動計算，
人類不斷追求更快、更準、更方便！在沒有
微處理機的時代，
人類就已經開始
想辦法讓計算
變得更快！

拓真 Takuma

熱愛科技・勇於挑戰

1642

Pascaline

帕斯卡計算機

Blaise Pascal
(1623-1662)

- 法國數學家帕斯卡為了協助父親進行稅務計算而發明。
- 可進行加法與減法。
- 採用齒輪與機械結構，屬於機械式計算機。

人類第一步
「讓計算機械化」

1944

Harvard Mark I

(IBM ASCC)

哈佛馬克一號



- 由美國哈佛大學與 IBM 合作開發。
- 採用電機機械式（繼電器、開關、齒輪等）。
- 長約 17 公尺、重約 5 噸，可進行自動序列計算。
- 主要用於軍事與科學計算。

計算速度大幅提升
但仍然是「機械+電機」

1940s-1950s

真空管電腦

Vacuum Tube Computer



ENIAC (1946)

- 世界上第一台通用電子計算機 (Electronic Digital Computer)。
- 使用約 18,000 顆真空管，佔地約 170 平方公尺，耗電量極大。
- 每秒可進行數千次加法運算，相較於機械式計算機快上數百倍。
- 主要用於軍事、彈道計算等科學應用。

進入電子計算時代
但體積龐大、耗電高、發熱嚴重雖然這些計算機
已經很厲害了，
但它們都又大又重，
而且速度、功耗和
可靠度都有
很大的限制！

玲奈 Reina

喜歡思考・樂於學習

小結：計算技術的演進

機械式計算
(1640s)

- 以齒輪等機械結構進行計算，速度慢，但開創了自動計算的概念。

電機機械式計算
(1940s)

- 使用繼電器、開關等電機元件，計算速度提升，但結構仍然龐大。

電子式計算（真空管）
(1940s-1950s)

- 以真空管進行電子計算，速度大幅提升，但體積大、耗電高、發熱嚴重，維護不易。

這一切，
都為了更小、更快，
更可靠的計算方式！
而下一個關鍵，就是——
半導體技術的出現！

1-4

電腦曾經大到塞滿一個房間

Computers Used to Fill an Entire Room

在微處理機出現之前，電腦需要整個房間才能運作！

拓真 Takuma

熱愛科技・勇於挑戰

ENIAC (1946)

世界上第一台通用電子計算機
Electronic Numerical Integrator
and Computer



- 使用約 18,000 顆真空管
- 佔地約 170 平方公尺（約一個籃球場）
- 重量超過 30 公噸
- 每秒可進行約 5,000 次加減法運算
- 主要應用於軍事計算（如彈道軌跡）

巨大、耗電、發熱嚴重

TRADIC (1954)

全球第一台使用電晶體的電腦
TRAnsistorized Digital Computer



- 使用約 800 顆電晶體
- 體積與大型機相當，仍需整個房間
- 可靠度比真空管電腦高
- 功耗與發熱大幅降低
- 主要應用於美國空軍的防空系統

體積仍大，但更可靠

大型電腦的共同特點



需要整個房間

佔地數十至數百平方公尺



耗電量大

需專用電力系統



發熱嚴重

需要強力冷卻設備



維護成本高

需要專業人員維護



運算速度受限

相較現代晶片非常緩慢



想一想...



為什麼早期的電腦
必須這麼大？
如果沒有半導體技術，
現在我們的手機、電腦
和 AI 還會存在嗎？



現在一顆小小的晶片，
可以有這麼強的能力，
你能想像嗎？

現在這些龐大的
計算能力，
竟然可以縮進
一顆晶片裡，
而且更強大！

從一個房間
到一顆晶片，
這是科技的
巨大飛躍！

1-5

關鍵轉折：半導體

The Turning Point : Semiconductor

從真空管到一顆晶片，
這是改變世界的技術革命！

現在，
一顆指甲大小的晶片，
就能擁有超強的
計算能力，
這就是半導體
的魅力！

科技的進步，
就是不斷讓
計算能力變得
更小、更快、
更可靠！

拓真 Takuma

熱愛科技・勇於挑戰

真空管 (Vacuum Tube)

~ 1950s



- 體積龐大
- 功耗高
- 發熱嚴重
- 壽命短
- 可靠度低

電晶體 (Transistor)

1950s - 1960s



- 體積大幅減小
- 功耗降低
- 發熱嚴減少
- 壽命較長
- 可靠度提升

積體電路 (Integrated Circuit, IC)

1960s - 1970s



- 多個電晶體整合
在同一晶片
- 體積更小
- 成本降低
- 可靠度提高
- 開始大規模應用

大規模積體電路 (LSI)

1970s - 1980s



- 數千到數萬個電晶體
整合在同一晶片
- 功能更強大
- 體積更小
- 成本進一步降低
- 應用領域大幅擴展

微處理機 (Microprocessor)

1970s 以後



- 將整個中央處理單元
(CPU) 整合在單一 IC
- 高效能、低功耗
- 體積極小
- 高可靠度
- 開啟個人電腦、
嵌入式系統與智慧裝置
的時代

玲奈 Reina

喜歡思考・樂於學習



半導體帶來的關鍵改變



體積更小

從整個房間 → 一顆晶片



功耗更低

更省電、更環保



可靠度更高

不易故障、壽命更長



成本更低

大量生產、普及應用



效能更強

運算能力持續提升



小知識



微處理機之所以能實現，
是因為半導體製程的不斷進步，
讓數十萬、數百萬，甚至數十億
個電晶體都能整合在一顆晶片上。
這不僅改變了電腦，也徹底改變
了我們的生活！

小小晶片，
大大改變世界！

半導體，讓不可能成為可能！

1-6

什麼是微處理機？

Microprocessor

一顆小小的晶片，
掌控整個世界的運作！

Small Chip
Big Possibilities

微處理機就像是
電腦的大腦，
負責接收指令、
思考處理，並輸出
結果，讓各種裝置
動起來！

雖然體積很小，
但它卻能執行
數十億次的運算，
是現代科技
不可或缺的核心！

拓真 Takuma

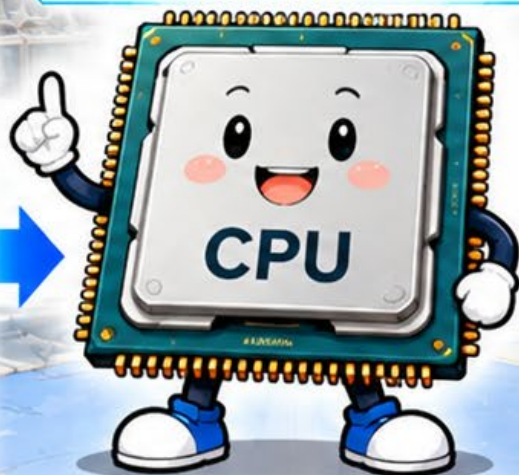
熱愛科技・勇於挑戰

輸入 (Input)
接收資料與指令



處理 (Process)

執行指令、進行運算、控制各單元



輸出 (Output)
產生處理結果



玲奈 Reina

喜歡思考・樂於學習

微處理機的定義

微處理機 (Microprocessor)
是將中央處理單元 (CPU) 製作在
單一積體電路 (IC) 上的可程式化
處理器，能夠接收指令、進行運算，
並控制周邊裝置，是電腦系統的核心。



一顆晶片，
無限可能！

微處理機的主要功能

- 1 執行指令 (Instruction Execution)
- 2 進行數學與邏輯運算 (Arithmetic & Logic)
- 3 控制資料的流動 (Control)
- 4 與記憶體及周邊裝置進行資料交換
- 5 協調整個系統的運作

常見的應用

從個人電腦到智慧裝置，微處理機無所不在！



微處理機，讓生活更智慧！

1-7

1971 : Intel 4004

The First Commercial Microprocessor

一顆小小的晶片，
開啟了個人計算的時代！

你知道嗎？
現在電腦的大腦
一開始竟然是為
了計算機而誕生的！

拓真 Takuma

熱愛科技・勇於挑戰

1 Canon 的需求 計算機 Canola 130



- 1969 年，日本 Canon 公司正在開發桌上型計算機 Canola 130。
- 需要一個小型、低成本、可程式化的計算控制晶片。

2 Busicom 的合作

Busicom



東京・Busicom 公司與 Intel 團隊
共同開發計算機控制晶片。

- Busicom 提出需求規格。
- Intel 工程師 Ted Hoff 等人，發現可以將整個 CPU 做在一顆晶片上。
- 這是微處理機誕生的關鍵契機！

3 Intel 4004 的誕生 (1971)



一顆晶片，
改變了世界！

- 1971 年，Intel 發表全球第一顆商業化微處理機 Intel 4004。
- 4-bit、時脈約 740 kHz。
- 約 2,300 個電晶體。
- 可執行加、減、邏輯、控制等指令。
- 主要應用於計算機，但開啟了微處理機時代。

從這顆 4-bit
的小晶片開始，
人類的計算能力
走出了實驗室，
走進了日常生活！

玲奈 Reina

喜歡思考・樂於學習

Intel 4004 的基本規格

發表年份	1971
資料位元數	4-bit
時脈頻率	約 740 kHz
電晶體數	約 2,300 個
製程	10 μ m PMOS
指令集	約 46 條指令
主要應用	計算機 (如 Canon Canola 130)

雖然只有 4-bit，
但它是所有現代
微處理機的起點！



Intel 4004 的意義

- 1 全球第一顆商業化微處理機
- 2 將整個 CPU 整合在單一晶片上
- 3 開啟了個人計算與嵌入式系統的時代
- 4 奠定後續 8-bit、16-bit、32-bit... 微處理機的發展基礎

想一想...

- 1 如果當時沒有 Intel 4004，你覺得計算機還會怎麼發展？
- 2 為什麼從計算機開始，最後卻發展出電腦、手機、甚至各種智慧裝置？

一顆為計算機而生
的晶片，為什麼會
改變整個世界？



1-8

4-bit → 8-bit：微處理機開始長大

From 4-bit to 8-bit: A New Generation of Microprocessors

更強的運算能力，
開啟更多應用的可能！8-bit 讓
微處理機走出實驗室，
進入我們的生活，
從電腦、遊戲到工業
設備，都能看到
它的身影！4-bit 雖然是
重要的起點，
但 8-bit 的出現，
讓微處理機真正
進入實用與普及
的時代！

拓真 Takuma

熱愛科技・勇於挑戰

Intel 8008
(1972)Intel 首款
8-bit 微處理機Intel 8080
(1974)推動個人電腦
發展的重要晶片Zilog Z80
(1976)廣泛應用於
電腦、遊戲機
與嵌入式系統Motorola
6800 / 6809 / 68HC11
(1974 / 1978 / 1990)工業控制
汽車電子
嵌入式應用RCA CDP1802
(1976)簡潔的設計
廣泛應用於
教育與嵌入式系統

玲奈 Reina

喜歡思考・樂於學習



從 4-bit 到 8-bit 的進步

4-bit 時代
(例：Intel 4004)

- 4-bit 資料寬度
- 運算能力有限
- 主要用於計算機與簡單控制

8-bit 時代
(例：Intel 8080, Z80)

- 8-bit 資料寬度
- 更高的運算能力
- 記憶體位址空間大幅提升
- 應用範圍更廣



8-bit 微處理機的典型應用

個人電腦
(如 Altair 8800)遊戲機
(如任天堂 FC)工業控制
(如 PLC)汽車電子
(如引擎控制)

...



想一想...

- 1 為什麼 8-bit 能帶來更強的運算能力？
- 2 如果沒有這些 8-bit 微處理機，現在的個人電腦、遊戲機或工業設備會是什麼樣子？
- 3 你還知道哪些 8-bit 微處理機嗎？

下一步，
會不會出現
16-bit 呢？

每一個位元的進化，都推動世界向前！

1-9

16-bit 與 PC 時代來臨

The Arrival of 16-bit and the PC Era

從實驗室走進家庭與辦公室，
微處理機讓電腦走入每個人的生活！

A Personal
Computer
A Brighter Future

8088 很特別！
內部是 16-bit，
但外部資料匯流排
只有 8-bit，
這樣既能降低成本，
又能相容 8-bit 裝置。

拓真 Takuma

熱愛科技・勇於挑戰

1 Intel 8088 (1979)

內部 16-bit・外部 8-bit 的特殊設計



16-bit 的
實力，
8-bit 的成本！

- 內部架構為 16-bit
- 外部資料匯流排為 8-bit
- 可相容 8-bit 周邊裝置
- 為後來 IBM PC 採用的關鍵處理器

2 IBM PC 5150 (1981)

開啟個人電腦的新紀元



從實驗室的
技術，
變成每個人
都能使用的電腦！

- 採用 Intel 8088 處理器 (4.77 MHz)
- 搭配 MS-DOS 作業系統
- 建立個人電腦的標準架構
- 帶動軟體、生態系與周邊產業的快速發展

3 重要規格比較

項目	Intel 8088	IBM PC 5150
推出年份	1979	1981
資料寬度 (內部/外部)	16-bit / 8-bit	16-bit / 8-bit (使用 8088)
時脈頻率	5-10 MHz	4.77 MHz
位址匯流排	20-bit (可定址 1 MB)	20-bit (最大 640 KB)
主要應用	微處理器	個人電腦

IBM PC 5150
不只是一台電腦，
更是一個全新的
開始！
它讓程式設計、
辦公應用、遊戲、
教育都進入
家庭與學校！

玲奈 Reina

喜歡思考・樂於學習

4 16-bit 時代的重要處理器



Intel 8086
(1978)
16-bit



Intel 8088
(1979)
16-bit / 8-bit



Motorola
68000
(1979)
16/32-bit



Zilog Z8000
(1979)
16-bit

16-bit 讓微處理機具備更強的運算能力與更大的應用空間！

5 PC 時代帶來的改變

辦公應用

文書處理
試算表

教育學習

電腦教學
程式設計

遊戲娛樂

家用遊戲
多媒體應用

產業發展

推動軟體產業
與周邊設備
蓬勃發展

PC 不只是電腦，
更開啟了數位時代的大門！

想一想...

- 1 為什麼 Intel 8088 要採用「內部 16-bit、外部 8-bit」的設計？
- 2 如果 IBM PC 5150 當時沒有使用 8088，而是使用 8086，可能會有什麼不同？
- 3 16-bit 微處理機的出現，對我們現在的生活有哪些影響？

如果沒有
這一步，
現在的電腦世界
會不會不一樣？



1-10

32-bit → 64-bit

From 32-bit to 64-bit : More Power, More Possibilities

位元數不只是數字，
而是打開更多應用的大門！

Faster
Smarter
For a Better
Tomorrow

從 32-bit 到 64-bit，
不只是速度變快，
更能處理更大的資料，
與更複雜的應用，
讓電腦走進更廣闊
的世界！

Motorola MC68000
(1979)

**32-bit**

- 32-bit 資料寬度
- 68000 系列
- 廣泛應用於
個人電腦與嵌入式系統

32-bit 時代的先驅

Intel 80386
(1985)

**32-bit**

- 32-bit 資料寬度
- 首次引入保護模式
- 支援虛擬記憶體
- 開啟多工作業時代

進入 32-bit 時代

Intel 80486
(1989)

**32-bit**

- 32-bit 資料寬度
- 內建快取記憶體
- 整合數學協處理器
- 提升整體效能

效能大幅提升

Intel Pentium
(1993)

**32-bit**

- 更高的時脈
- 超純量架構
- 多媒體指令 (MMX)
- 應用更廣泛

多媒體與網路應用

64-bit 時代
(2003 之後)

**64-bit**

- 64-bit 資料寬度
- 可尋址更大記憶體
- 更高的運算效能
- 支援現代作業系統
與 AI 應用

迎向未來

64-bit 讓電腦
能處理更大的資料，
支援 AI、影像處理、
科學計算等應用，
也讓我們的生活
更加便利與
智慧化！

拓真 Takuma

熱愛科技・勇於挑戰

玲奈 Reina

喜歡思考・樂於學習

位元寬度的意義

位元寬度 (bit) 代表一次可處理的資料量，
影響可定址記憶體大小與運算能力。

位元數越大，
能處理的資料
就越多！！

8-bit	16-bit	32-bit	64-bit
$2^8 = 256$ (約 256 B)	$2^{16} = 65,536$ (約 64 KB)	$2^{32} = 4,294,967,296$ (約 4 GB)	2^{64} (理論上 16 EB)

位元數的進化跑道



更大的位元寬度，更遠的未來！

想一想...

- 1 為什麼 32-bit 時代能讓個人電腦普及？
- 2 64-bit 處理器相比 32-bit，最大的優勢是什麼？
- 3 在 AI、影像處理或大型遊戲中，為什麼需要 64-bit 處理器？

未來的
微處理機，
會是 128-bit 嗎？



1-11

Checkpoint ①：處理器為什麼一直增加位元數？

Why Do Processors Keep Increasing the Number of Bits?

位元數的增加，
不是為了炫技，
而是為了解決
更大的資料、更複雜
的應用與更廣闊
的未來！

拓真 Takuma

熱愛科技・勇於挑戰

4-bit
(1971)計算機
簡單控制

處理簡單資料

8-bit
(1972-1976)個人電腦
遊戲機

進入個人應用

16-bit
(1978-1980s)更大的記憶體
更豐富的作業系統

應用領域擴展

32-bit
(1985-1990s)多工處理
圖形化介面
網路應用

進入多媒體與網路時代

64-bit
(2003-至今)巨量資料
AI・雲端運算
高效能計算

迎向智慧未來

位元數讓處理器
能一次處理更多的
資料，支援更大的
記憶體，運行更複雜
的程式，
也打開了 AI、雲端
與智慧生活
的大門！

玲奈 Reina

喜歡思考・樂於學習

為什麼要增加位元數？



處理更大的資料

位元數越多，一次可處理的資料量越大。



支援更大的記憶體

可定址的記憶體空間大幅提升。



提升運算能力

可處理更複雜的運算與資料結構。



支援更多應用

從文字、圖形、影音到 AI 與科學計算。



迎向未來需求

物聯網、雲端運算、AI 等需要更強的處理能力。

2 位元數提升帶來的改變

位元數越大，
能看到的世界越大！



位元數的進化，來自不斷擴大的想像力！

想一想...

- 為什麼 8-bit 處理器相較 4-bit 可以支援更複雜的應用？
- 如果今天的應用需要處理大量的影像或 AI 模型，為什麼必須使用 64-bit 處理器？
- 位元數的增加是否有缺點？（例如：成本、功耗、晶片面積）
- 未來會不會有比 64-bit 更大的處理器？你認為可能嗎？為什麼？

如果有
128-bit 呢？
會用在哪些
應用呢？



1-12

兩條重要道路：CISC vs. RISC

Two Major Paths: CISC vs. RISC (Instruction Set Architecture)

不同的設計哲學，
但都朝著相同的目標前進
——讓電腦更強大！

CISC 就像
一個工具箱，
工具很多，
可以用一條指令
完成複雜的工作！

拓真 Takuma

熱愛科技・勇於挑戰

CISC

複雜指令集架構

Complex Instruction Set Computer

功能豐富・一指多工

字串處理

資料搬移

乘除運算

輸入/輸出

複雜控制

.....

Instruction Set
Architecture
(指令集架構)

定義處理器
可以執行哪些
指令，以及
指令的格式與
執行方式。

RISC

精簡指令集架構

Reduced Instruction Set Computer

簡單精簡・動作俐落

載入/儲存

加減運算

邏輯運算

分支跳躍

.....

RISC 的工具較少，
但每個工具都很簡單
且高效率，
就像專業運動員一樣，
動作俐落、
速度更快！

玲奈 Reina

喜歡思考・樂於學習



什麼是指令集架構 (ISA) ？

指令集架構 (Instruction Set Architecture, ISA)
是一種介面規格，定義了：

- ✓ 處理器可以執行哪些指令
- ✓ 每條指令的格式與編碼方式
- ✓ 指令的執行方式與資料型態
- ✓ 程式設計與硬體實作之間的橋樑

不同的道路
相同的目標

更強大的電腦應用！



想一想...

- 1 為什麼會有 CISC 與 RISC 兩種不同的設計？
- 2 你覺得「指令多」一定比較好嗎？
- 3 除了速度之外，還有哪些因素會影響處理器的設計選擇？

如果你要設計
一個新的處理器，
你會選擇哪一條路？
為什麼？



1-13

CISC vs. RISC，一張圖看懂

A Clear Comparison: CISC vs. RISC

沒有絕對的好壞，
只有適合的應用場景！

Different
Architectures
A Brighter World

CISC 像是
「多功能瑞士刀」，
什麼都能做！
適合複雜的任務
與向下相容。



CISC

Complex Instruction Set Computer

多 (數百~數千)
例如：x86 具有豐富的指令集

可變長度
(1~15 位元組)

複雜
(需多階段解碼)

較複雜
(部分指令需多個時脈週期)

Intel x86 (8086, 8088, 80386,
80486, Pentium...) 等

個人電腦、伺服器
(重視軟體相容性與多樣應用)

比較項目

指令數量

指令長度

指令解碼

管線化
(Pipeline)

代表架構

典型應用

RISC

Reduced Instruction Set Computer

少 (通常數十~數百)
例如：RISC 約 30~200 條

固定長度
(多為 32 位元)

簡單
(易於快速解碼)

較容易
(大多為單一時脈週期完成)

MIPS, PowerPC, SPARC,
ARM, RISC-V 等

嵌入式系統、行動裝置、
網路設備 (重視效能與低功耗)



RISC 的指令雖然少，
但設計簡單、
動作俐落，
非常適合行動裝置
與嵌入式應用！

拓真 Takuma

熱愛科技・勇於挑戰

玲奈 Reina

喜歡思考・樂於學習

重點整理

- ✓ CISC 追求「功能豐富與相容性」
- ✓ RISC 追求「簡單高效與低功耗」
- ✓ RISC 較容易進行管線化設計
- ✓ 兩者各有優勢，依應用需求選擇
- ✓ 現代處理器也常結合兩者的優點
(例如：x86 內部採用 RISC-like 的微指令)

小結

CISC 與 RISC 代表了兩種不同的設計哲學：
CISC 強調「更多功能」、RISC 強調「更高效率」。
了解兩者的差異，有助於我們理解
不同處理器在生活中的應用場景，
也能更深入認識未來電腦系統的發展方向！

沒有最好，
只有最適合！

想一想...

- 1 你認為一般個人電腦多採用 CISC，
主要原因是什麼？
- 2 為什麼行動裝置多採用 RISC？
- 3 如果未來有一種新的應用需要
「超高效能 + 低功耗 + 高相容性」，
你會如何設計它的指令集？

不同的需求，
會選擇不同的
道路！



1-14

RISC 家族登場

Famous RISC Processors

不同的 RISC 處理器，
共同推動了電腦運算的進化！Simple Design
High Performance
Big Impact

RISC 並不是
只有 ARM！
在 1990 年代，
R4400、PowerPC
和 UltraSPARC II
都是非常重要的
RISC 代表！

拓真 Takuma

熱愛科技・勇於挑戰

MIPS R4400
(1996)MIPS
SIMPLE IS POWERFUL

- 64-bit RISC 處理器
- 採用 MIPS III 指令集
- 具備高效能的流水線架構
- 廣泛應用於工作站、伺服器及嵌入式系統

工作站
(Workstation)PowerPC 601
(1993)PowerPC
PERFORMANCE FOR A WIDER WORLD

- 32-bit RISC 處理器
- 由 IBM、Motorola、Apple 共同開發 (PowerPC 聯盟)
- 具備高效能與低功耗設計
- 廣泛應用於個人電腦、工作站與嵌入式系統

個人電腦
(Macintosh)UltraSPARC II
(1997)Sun
microsystems

- 64-bit RISC 處理器
- 採用 SPARC V9 指令集
- 高效能、多執行緒設計
- 主要應用於高階伺服器與企業級系統

伺服器
(Server)

這些 RISC 處理器
在不同的領域發光發熱，
從個人電腦到大型伺服器
都能看到 RISC 的身影！
它們為後來的 ARM
與更多行動裝置
奠定了基礎！

玲奈 Reina

喜歡思考・樂於學習

為什麼不同公司
都要發展自己的
RISC 處理器呢？

重點整理

- ✓ RISC 透過簡化的指令集設計，提升執行效率。
- ✓ MIPS R4400、PowerPC 601、UltraSPARC II 是 1990 年代具代表性的 RISC 處理器。
- ✓ 不同的 RISC 處理器應用於不同的領域，展現 RISC 架構的廣泛影響力。
- ✓ 這些技術也為後來的 ARM 與行動裝置發展鋪路。

小結

R4400、PowerPC 601 與 UltraSPARC II 代表了 RISC 架構在不同應用領域的成功，也顯示出「簡單指令、高效執行」的設計哲學，持續影響著現代處理器的發展！

不同的應用，
同一個精神！

想一想...

- 1 你覺得 RISC 架構適合哪些應用場景？為什麼？
- 2 如果要設計一個新的處理器，你會選擇 RISC 還是 CISC？
- 3 除了這三種，還有哪些著名的 RISC 處理器？它們應用在哪裡？



1-15

ARM：你每天都在用的架構

ARM: An Architecture You Use Every Day

ARM 的成功告訴我們：
好的架構設計
可以從小做起，
卻能改變全世界！

拓真 Takuma

熱愛科技・勇於挑戰



ARM 的誕生與發展

1978

Acorn Computers
在英國成立

Acorn

英國的一家
電腦公司

1985

開始研發
RISC 處理器ARM 1
第一顆 ARM 處理器
(1985)精簡指令、低功耗
適合嵌入式應用

1990

成立
ARM 公司

arm

Acorn、Apple、
VLSI Technology
共同成立 ARM 公司

1990s ~

授權 (Licensing) 模式
走向全球採用授權架構 (IP Licensing)
讓全球廠商都能使用 ARM
設計自己的晶片

ARM 的核心特色



精簡指令集 (RISC)

指令簡單、執行效率高



低功耗

非常適合行動裝置與嵌入式裝置



高效能 / 高能效比

以較少的電力完成更多工作



廣泛的生態系

全球眾多廠商支援與開發



授權模式

可被不同公司設計成各種晶片

Small Core
Big World
Efficient Processors
for a Smarter Life

你現在手上的
智慧型手機、平板，
幾乎都是 ARM！
ARM 不只是電腦，
更是推動現代生活
的隱形英雄！

玲奈 Reina

喜歡思考・樂於學習



ARM 無所不在



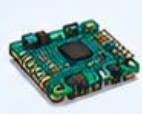
智慧型手機

iPhone
Android 手機

平板電腦

iPad
Android 平板

筆記型電腦

輕薄筆電
(如 MacBook)

嵌入式裝置

智慧家電
IoT 裝置

汽車電子

車用控制系統
ADAS

智慧手錶

穿戴式裝置
健康監測

網通設備

路由器
基地台

工業與機器人

工業控制
智慧製造

...

ARM 讓數十億個裝置更聰明，也讓我們的生活更便利！



想一想...

- 1 為什麼 ARM 特別適合
智慧型手機與行動裝置？
- 2 如果沒有 ARM，
我們的生活會有哪些不同？
- 3 你還知道哪些使用 ARM
的產品？
它們帶來了什麼好處？

除了手機，
你能找到 5 個以上
使用 ARM 的產品嗎？
想想看！



1-16

微處理機其實就在你身邊！

Microprocessors Are All Around You!

從手機到工廠設備，
微處理機已融入我們的日常生活，
讓世界更便利、更智慧！

Smart Devices
Smart Life
Smarter Future

別小看這顆小小的晶片！
微處理機已經在我們生活的每個角落默默工作，讓世界更便利！

拓真 Takuma

熱愛科技・勇於挑戰

智慧型手機 / 平板



應用：通訊、拍照、遊戲、AI 助理、各種 App

汽車電子



應用：引擎控制、車身電子、ADAS、車用娛樂系統

家電產品



應用：智慧控制、節能管理、物聯網連網、語音控制

智慧手錶 / 穿戴裝置



應用：健康監測、運動分析、通知提醒、行動支付

網路與通訊設備



應用：路由、交換、基地台、5G/光纖通訊、智慧家庭

機器人 / 自動化設備



應用：工業自動化、機器視覺、運動控制、智慧製造

工業與醫療設備



應用：精密控制、影像處理、即時監測、提升安全與效率

更多應用 ...



應用：無人機、智慧家居、安防監控、金融支付 ...

生活中看不見的微處理機，其實無所不在！只要有智慧功能的裝置，幾乎都少不了它的存在！

玲奈 Reina

喜歡思考・樂於學習

還有哪些裝置沒有提到呢？
你能再舉出幾個例子嗎？

重點整理

- ✓ 微處理機已廣泛應用於生活中的各種裝置。
- ✓ 它讓裝置具備智慧化、自動化與連網能力。
- ✓ 從消費性電子到工業控制，微處理機持續推動社會進步。
- ✓ 未來隨著 AI 與 IoT 的發展，應用領域將更加廣泛。

生活中的隱形英雄

下次使用手機、開車、或操作家電時，別忘了感謝這顆默默工作的微處理機！

看不見，但無所不在！

想一想...

- 1 你今天使用了哪些包含微處理機的裝置？
- 2 你覺得沒有微處理機的生活會變成什麼樣子？
- 3 未來你希望微處理機能帶來哪些新的應用？

1-17

行動裝置 × 智慧居家 × IoT × AI

Mobile Devices × Smart Home × IoT × AI

微處理機讓萬物更智慧，
連結我們的現在與未來！Technology
Connects People
for a Better
Tomorrow!從手機到智慧家庭，
從 IoT 到 AI，
這些看起來不同的
應用，其實都離不開
微處理機的力量！

行動裝置

隨時隨地，連結世界

智慧型手機、平板、穿戴裝置等
都內建微處理機，提供強大的
運算能力與豐富的應用服務。

智慧居家

更舒適・更安全・更節能

透過微處理機與網路連結，
智慧居家設備能自動控制、遠端管理，
讓生活更便利、更安全、更節能。

物聯網與智聯網 (IoT)

萬物連網・數據驅動

各種裝置透過網路互相連結，
收集與分享資料，實現智慧城市、
智慧製造、智慧交通、智慧醫療等
多元應用。

人工智慧 (AI)

讓裝置更聰明，讓生活更美好

微處理機提供強大的運算能力，
讓 AI 技術能在各種裝置上實現，
從語音辨識到影像分析，
創造更多可能！AI 的大腦
也是由微處理機
來執行運算！
從語音助理到
自動駕駛，
都讓我們的生活
更便利、更智慧！

拓真 Takuma

熱愛科技・勇於挑戰

玲奈 Reina

喜歡思考・樂於學習



重點整理

- ✓ 行動裝置、智慧居家、IoT 與 AI 都需要微處理機。
- ✓ 微處理機讓裝置具備運算、控制、連網與智慧化的能力。
- ✓ 這些應用正在改變我們的生活、工作與社會。
- ✓ 未來將有更多裝置透過 AI 與 IoT 互聯，創造更大的價值。



小結

微處理機不只存在於電腦中，
它已經融入我們生活的每一個角落，
讓世界變得更便利、更安全、更智慧！小小的晶片，
大大的改變！

想一想...

- 1 你每天會使用哪些內建微處理機的裝置？
- 2 你認為未來還有哪些日常物品
會加入微處理機與 AI 的功能？
- 3 這些技術會如何改變你的生活？
對社會可能帶來什麼影響？



1-18

摩爾定律 Moore's Law

More Transistors, Smaller Chips, Greater Possibilities

小小的晶片，
推動了巨大的科技進步！

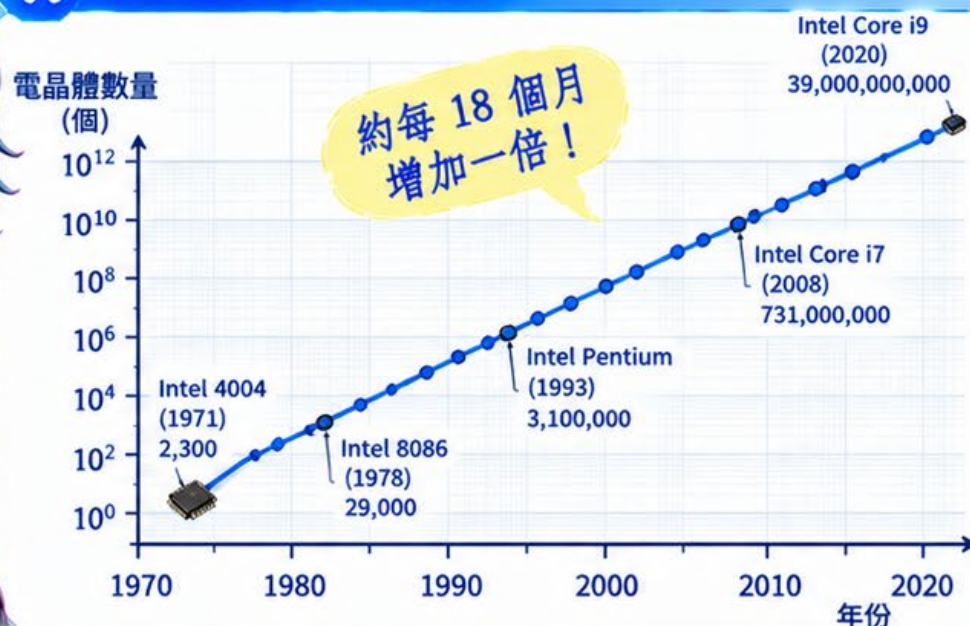
Smaller Chips
Bigger Ideas
Brighter
Tomorrow!

摩爾定律告訴我們：
晶片上的電晶體數量
持續快速增加，
讓電腦越來越強大，
而且越來越小！

拓真 Takuma

熱愛科技・勇於挑戰

★ 電晶體數量的成長趨勢



★ 教材中的摩爾定律重點

- 1 晶片上可容納的電晶體數量約每 18 個月增加一倍。
- 2 在相同面積的晶片上，可容納的電晶體數量約每 18 個月增加一倍。
- 3 在相同的晶片面積下，每一個電晶體的尺寸約每 18 個月縮小一半。

因為電晶體
越來越多、
尺寸越來越小，
微處理機的效能
越來越強，
也讓更多創新的
應用成為可能！

玲奈 Reina

喜歡思考・樂於學習

⚙️ 重點整理

- ✓ 摩爾定律描述電晶體數量的指數成長。
- ✓ 約每 18 個月，晶片上的電晶體數量會增加一倍。
- ✓ 在相同面積下，可容納的電晶體數量持續提升。
- ✓ 在相同面積下，每個電晶體的尺寸約每 18 個月縮小一半。
- ✓ 這使得微處理機的效能提升、體積縮小、應用更廣泛。

💡 小結

摩爾定律不僅是技術的進步，
更是推動整個資訊科技產業發展的重要力量。
晶片越來越小、越來越強大，
讓我們的生活變得更加便利，
也為未來帶來無限的可能！

小小晶片，
大大改變
世界！

🧠 想一想...

- 1 為什麼電晶體數量增加，可以提升電腦的效能？
- 2 晶片尺寸越來越小，對我們的生活有什麼好處？
- 3 你認為摩爾定律在未來還會持續嗎？如果趨勢放緩，可能的影響是什麼？



1-19

微處理機的下一站？

What's Next? The Future of Microprocessors

Smarter Technology
Brighter People
Better Tomorrow科技持續進化，
未來的運算世界充滿無限可能！Beyond
Processors
A Smarter
Tomorrow !

微處理機的發展
不會停止！
除了更強大的 AI，
還有平行運算與
量子電腦等新技術，
正在開創
下一個世代！

拓真 Takuma

熱愛科技・勇於挑戰



AI 處理器 (AI Processor)

專為人工智慧而生的運算核心



- 針對深度學習、機器學習進行最佳化
- 具備大量平行運算單元 (如 GPU / TPU / NPU)
- 廣泛應用於影像辨識、語音處理、自動駕駛、生成式 AI 等
- 讓 AI 從雲端走向邊緣裝置，貼近我們的生活



生成式 AI



自動駕駛



智慧製造



AI 手機



平行運算 (Parallel Computing)

同時處理更多，加速運算效能



- 利用多核心、多處理器同時進行運算
- 大幅提升資料處理速度與效率
- 應用於科學計算、氣候模擬、大數據分析、AI 訓練等
- 從單核心走向多核心、超級電腦，讓複雜問題更快解決



高效能運算



大數據分析



雲端運算



智慧城市



量子電腦 (Quantum Computing)

顛覆傳統運算的全新可能



- 利用量子位元 (Qubit) 進行運算
- 可同時計算多種狀態，具有指數級的運算潛力
- 在密碼破解、材料設計、藥物研發、優化問題等領域具重大應用
- 目前仍在研發階段，但已被視為下一世代運算的重要方向



密碼安全



新藥研發



材料設計



最佳化問題

未來的微處理機
不只是更小、更快，
而是結合 AI、
平行運算與量子技術，
讓我們的生活、
工作與社會
變得更加智慧
與美好！

玲奈 Reina

喜歡思考・樂於學習



重點整理

- ✓ AI 處理器、平行運算與量子電腦是微處理機發展的重要方向。
- ✓ AI 處理器讓智慧應用更普及，平行運算提升運算效率，量子電腦則帶來顛覆性的可能。
- ✓ 這些新技術將推動各行各業的創新與發展。
- ✓ 未來的運算世界將更加多元，為人類社會創造更大的價值。



小結

微處理機的下一站，
不只是更小、更快，
而是結合 AI、平行運算與量子電腦，
開啟更多未知的可能，
讓世界變得更智慧、更美好！

未來，
由你我一起
創造！



想一想...

- 1 你最期待未來微處理機的哪一種發展？為什麼？
- 2 你認為 AI、平行運算或量子電腦會對你的生活帶來什麼改變？
- 3 除了課本中提到的應用，你還能想到哪些新的可能性？



1-20

從外面看完，現在打開 CPU！

Let's Open the CPU!

小小的晶片裡，
藏著一個龐大的世界！

Explore
Inside the CPU
Explore
a Bigger World!

終於要打開
CPU 了！
讓我們看看
裡面到底
有什麼吧！

拓真 Takuma

熱愛科技・勇於挑戰

金屬散熱蓋
(Heat Spreader)

晶片核心
(DIE)

封裝基板
(Package Substrate)

等等！
CPU 不能
真的拆啦！
這只是示意圖！
實際的 CPU
非常精密，
一旦拆開就
會損壞，
無法再使用！

請勿自行拆解
實際的 CPU！

玲奈 Reina

喜歡思考・樂於學習

重點整理

- ✓ CPU 的外觀很小，但內部包含極為複雜的電路。
- ✓ 晶片核心 (Die) 是真正進行運算的地方。
- ✓ CPU 由多層結構組成，包括散熱蓋、晶片核心與封裝基板。
- ✓ CPU 製程非常精密，一般使用者不應自行拆解。
- ✓ 接下來我們將進一步認識 CPU 的內部架構與各個組成元件。

小結

雖然 CPU 的外觀看起來只是一個小小的晶片，但打開後你會發現，裡面有極其精密的結構與數以億計的電晶體。接下來，讓我們一起深入了解 CPU 的內部世界吧！

小小一顆，
蘊藏無限
可能！

想一想...

- 1 你覺得 CPU 內部會有哪些重要的元件？
- 2 為什麼 CPU 不適合讓一般使用者自行拆解？
- 3 如果沒有散熱蓋，CPU 會發生什麼問題？



1-21

微處理機的基本架構

Basic Architecture of a Microprocessor

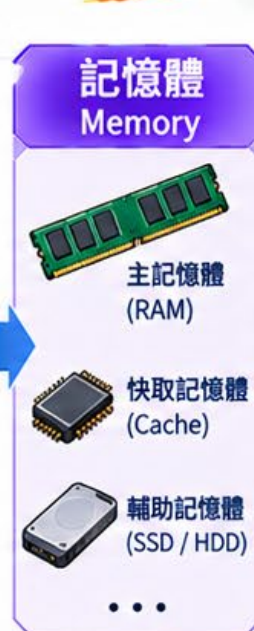
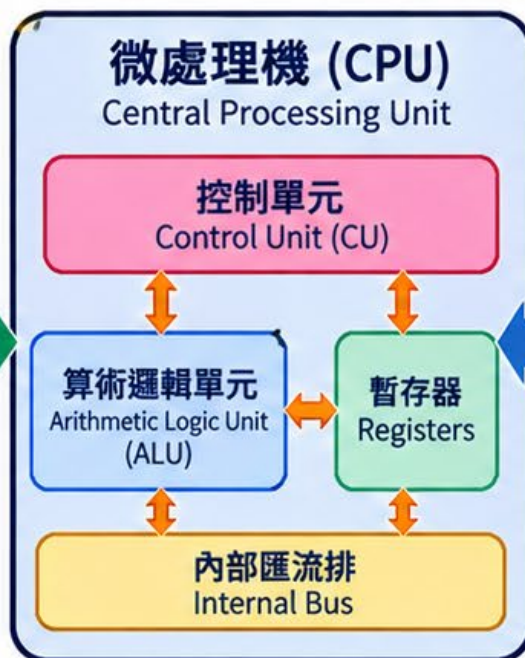
輸入資訊・處理資料・輸出結果
微處理機是電腦系統的核心！

From Input
to Output,
Everything
Starts Here! 😊

微處理機的工作
可以用三個步驟來
理解：輸入、處理、
輸出。它會接收外界
的資料，經過運算
與控制後，輸出
我們需要的結果！

拓真 Takuma

熱愛科技・勇於挑戰



微處理機就像
電腦的大腦，透過
輸入、處理、輸出
來完成各種任務，
讓我們的生活
更加便利！

玲奈 Reina

喜歡思考・樂於學習



重點整理

- ✓ 微處理機的基本架構包含：輸入、CPU、記憶體與輸出。
- ✓ CPU 由控制單元 (CU)、算術邏輯單元 (ALU)、暫存器 (Registers) 及內部匯流排 (Internal Bus) 組成。
- ✓ 記憶體用來儲存程式、資料與運算結果。
- ✓ 輸入裝置將外界的資料轉換為電腦可以處理的數位訊號。
- ✓ 輸出裝置將處理後的結果呈現給使用者。



小結

微處理機是整個電腦系統的核心，
透過輸入、處理、輸出的循環，
與記憶體及各種周邊裝置互相合作，
才能完成我們日常生活中
各種豐富的應用！

小小的架構，
創造大大的
可能！



想一想...

- 1 你日常生活中使用的哪些裝置屬於輸入裝置？哪些屬於輸出裝置？
- 2 為什麼需要記憶體來暫存資料？如果沒有記憶體會發生什麼事？
- 3 你認為未來的微處理機在架構上還可能有哪些改進？



1-22

CPU 裡住著誰？

Who Lives Inside the CPU?

這些成員分工合作，
讓電腦接收、處理、儲存並輸出資料！

Different
Parts
A Powerful
CPU !

CPU 不是只有一個晶片而已，
裡面有很多
重要的成員，
大家分工合作，
才能完成
各種任務！

拓真 Takuma

熱愛科技・勇於挑戰



雖然每個成員
各有不同的工作，
但它們透過
內部匯流排
互相連結，
才能快速交換資料，
讓 CPU 高速、
有效率地運作！

玲奈 Reina

喜歡思考・樂於學習



重點整理

- ✓ CPU 由輸入單元 (I/O)、輸出單元 (I/O)、內部匯流排、暫存器、程式計數器 (PC)、控制單元 (CU) 及算術邏輯單元 (ALU) 所組成。
- ✓ 各單元分工合作，共同完成指令的執行。
- ✓ 內部匯流排負責在各單元之間傳送資料、位址與控制訊號。
- ✓ CPU 是電腦系統的核心，決定整體運算效率與功能。



小結

CPU 裡的每一個成員都有重要的任務，
只有透過良好的分工與協同合作，
才能讓電腦快速、正確地完成
各種資料的處理。
了解這些基本組成，
是學習電腦硬體的第一步！

小小晶片，
大大能量！



想一想...

- 1 你認為在 CPU 的各個單元中，
哪一個最重要？為什麼？
- 2 如果沒有暫存器，CPU 的運作
會受到什麼影響？
- 3 你能想像內部匯流排在生活中
類似什麼角色嗎？



1-23

ALU：CPU 的計算高手

ALU：The Computation Expert in CPU

不論是加減乘除，
還是邏輯判斷，ALU 都難不倒！

Calculate
Logic
Decide
Enable a Smarter
World!

ALU 就像
CPU 裡的計算高手，
負責各種數學運算
和邏輯判斷，
是電腦能夠思考
與決策的重要關鍵！

交給我吧！
計算與邏輯，
我都行！

輸入 A
(Input A)

0101 1100

輸入 B
(Input B)

0011 0101

ALU

運算結果
(Result)

1001 0001

ALU 不只能做
加減乘除，
還能進行邏輯運算，
例如 AND、OR、
NOT 等運算。
這些運算讓電腦
能夠進行判斷，
完成各種任務！



算術運算 (Arithmetic Operations)

+ 加法
(Addition)

5 + 3 = 8

- 減法
(Subtraction)

7 - 2 = 5

× 乘法
(Multiplication)

4 × 3 = 12

÷ 除法
(Division)

8 ÷ 2 = 4



邏輯運算 (Logic Operations)

AND
且運算

0 AND 0 = 0
0 AND 1 = 0
1 AND 0 = 0
1 AND 1 = 1

OR
或運算

0 OR 0 = 0
0 OR 1 = 1
1 OR 0 = 1
1 OR 1 = 1

NOT
非運算

NOT 0 = 1
NOT 1 = 0

XOR
互斥或運算

0 XOR 0 = 0
0 XOR 1 = 1
1 XOR 0 = 1
1 XOR 1 = 0

拓真 Takuma

熱愛科技 · 勇於挑戰

玲奈 Reina

喜歡思考 · 樂於學習



重點整理

- ✓ ALU (算術邏輯單元) 是 CPU 中負責進行算術運算與邏輯運算的核心元件。
- ✓ 算術運算包括加、減、乘、除等基本運算。
- ✓ 邏輯運算包括 AND、OR、NOT、XOR 等，用於判斷與控制。
- ✓ ALU 接收來自暫存器的資料，運算後將結果送回暫存器。
- ✓ ALU 與控制單元 (CU) 協同運作，依指令進行正確的運算。



小結

ALU 是 CPU 的計算高手，
能夠快速完成各種數學運算與邏輯判斷。
有了 ALU，電腦才能進行資料處理，
做出正確的判斷，
並執行各種應用程式，
讓我們的生活更加便利！

小小的運算，
創造大大的
可能！



想一想...

- 1 為什麼電腦需要同時具備算術運算與邏輯運算的能力？
- 2 在日常生活中，你覺得有哪些應用是使用到 ALU 的運算？
- 3 如果沒有 ALU，電腦還能完成哪些任務？哪些任務將無法進行？



1-24

Register : CPU 的超高速工作桌

Register : The Ultra-Fast Workspace of the CPU

資料先放在暫存器，
運算更快速！Small
Registers
Big
Possibilities!Register 就像
CPU 的工作桌！
把需要的資料
先放在這裡，
運算時可以
立刻使用，
速度超快！

拓真 Takuma

熱愛科技・勇於挑戰

Register 的特點

- ✓ 速度非常快
- ✓ 容量很小
- ✓ 由多個暫存器組成
- ✓ 存放正在使用的資料、位址或運算結果

需要的資料
我先放在
Register，
運算馬上開始！

Register

快速運算！

Memory 與 Register 的差別

Register (暫存器)



- ✓ 速度：非常快
- ✓ 容量：很小
- ✓ 位置：在 CPU 內部
- ✓ 用途：暫存正在使用的資料

Memory (記憶體)



- ✓ 速度：較慢
- ✓ 容量：很大
- ✓ 位置：在 CPU 外部
- ✓ 用途：長期儲存程式與資料

暫存器就像
桌上的小抽屜，
雖然容量不大，
但拿取速度超快！
先把常用的資料
放在這裡，
運算效率
大大提升！

玲奈 Reina

喜歡思考・樂於學習



重點整理

- ✓ Register (暫存器) 位於 CPU 內部，是 CPU 的超高速工作桌。
- ✓ 暫存器容量很小，但讀寫速度非常快。
- ✓ CPU 會將正在使用的資料、位址或運算結果暫存在暫存器中。
- ✓ 暫存器由多個暫存器組成，例如 R0、R1、R2...
- ✓ 與記憶體相比，暫存器速度快但容量小，兩者在電腦系統中扮演不同的角色。



小結

暫存器是 CPU 內部的高速儲存空間，雖然容量不大，但對 CPU 進行運算與資料處理非常重要。有了暫存器，CPU 才能快速取得資料，完成各種指令，讓我們的電腦運作更順暢！

小小的抽屜，
大大的效率！

想一想...

- 1 為什麼暫存器的速度比記憶體快？
- 2 如果暫存器容量很大，會有什麼問題？
- 3 你認為 CPU 裡的暫存器在日常生活中像什麼東西？



1-25

PC：下一條指令在哪裡？

PC：Where is the Next Instruction?

PC 就像導航員，
告訴 CPU 下一條指令在哪裡，
讓程式一步一步向前走！

Good
Addresses 😊
Lead to
Great Results!

PC (Program Counter) 就像導航員，負責記錄下一條要執行的指令位址！

拓真 Takuma

熱愛科技・勇於挑戰

什麼是 PC？

- ✓ PC (Program Counter) 是 CPU 內的一個暫存器。
- ✓ 它存放「下一條要執行的指令位址」。
- ✓ CPU 取出指令後，PC 會自動 +1，指向下一條。
- ✓ 當有跳躍 (Jump) 或呼叫 (Call) 指令時，PC 會被改寫為新的位址。

PC 的工作：
永遠指向下一步！

下一站
0x0001！
出發！

PC

Program Counter
(程式計數器)

0x0000

0x0001

0x0002

...

記憶體 (Memory)

位址 (Address)	指令 (Instruction)
0x0000	LOAD A, 10
0x0001	ADD A, 20
0x0002	STORE A, 30
0x0003	..
⋮	⋮

CPU 會依照 PC 指示的位址，從記憶體中取出指令來執行！

只要 PC 一直正確地指向下一條指令，電腦就能按照程式的順序一步一步完成各種工作！

玲奈 Reina

喜歡思考・樂於學習



重點整理

- ✓ PC (Program Counter) 是 CPU 內的暫存器，用來存放下一條要執行的指令位址。
- ✓ CPU 取出指令後，PC 會自動 +1。
- ✓ 遇到跳躍 (Jump)、呼叫 (Call) 或中斷 (Interrupt) 時，PC 會被改寫為新的位址。
- ✓ PC 的位址寬度決定 CPU 可定址的記憶體範圍 (例如 16 位元可定址 64 KB)。



小結

PC 雖然只是一個小小的暫存器，卻是 CPU 能夠持續執行程式的關鍵。它就像導航員，指引著 CPU 一步一步前進，讓電腦的世界按照程式的順序有條不紊地運作！

有了 PC，
程式才能
向前走！



想一想...

- 1 如果 PC 目前的內容是 0x0005，且執行完一條普通指令後，PC 會變成多少？
- 2 當程式執行跳躍 (Jump) 指令時，PC 的內容會發生什麼變化？
- 3 為什麼需要 PC？如果沒有 PC，CPU 會遇到什麼問題？



1-26

Control Unit: CPU 的指揮官

Control Unit: The Commander of CPU

控制單元發出控制訊號，
協調各單元動作，
讓 CPU 有條不紊地工作！

Good Control
Makes
Everything
Work!

Control Unit
就像 CPU 的指揮官，
負責產生各種
控制訊號，
協調各單元的動作，
讓整個 CPU
能正確地執行
指令！

拓真 Takuma

熱愛科技・勇於挑戰

各就各位！
依照指令，
開始動作！

控制訊號的例子：

- ✓ 讀取 / 寫入 (Read / Write)
- ✓ 選擇資料來源 (Select Source)
- ✓ 選擇運算功能 (ALU Operation)
- ✓ 控制資料流向 (Data Transfer)
- ✓ 協調時序 (Timing Control)
- ✓ 處理中斷 (Interrupt Control)

沒有
Control Unit
的協調，
CPU 的各個單元
就像一群各做各的
無法合作，
電腦也就
無法正常運作了！



玲奈 Reina

喜歡思考・樂於學習



重點整理

- ✓ Control Unit (CU) 是 CPU 的指揮官，負責產生各種控制訊號，協調各單元的動作。
- ✓ CU 會先解碼指令 (Instruction Decode)，再依指令內容產生相對應的控制訊號。
- ✓ 控制訊號用來控制 PC、暫存器、ALU、記憶體與 I/O 的運作。
- ✓ 沒有 Control Unit，各單元無法協同工作，CPU 就無法執行程式。



小結

Control Unit 是 CPU 的大腦指揮官，它解讀指令、發出控制訊號，讓 PC、暫存器、ALU、記憶體與 I/O 能夠依序且正確地工作。有了 CU 的協調，CPU 才能像一個團隊，高效率地執行各種任務！

指揮得好，
電腦才不會
亂跑！



想一想...

- 1 你認為 Control Unit 主要的工作是什麼？
- 2 如果沒有 Control Unit，CPU 的各個單元會發生什麼情況？
- 3 你覺得為什麼需要控制訊號來協調各單元，而不是讓各單元自行運作？



1-27

Internal Bus : CPU 裡的高速公路

Internal Bus : The Superhighway Inside CPU

資料、指令、控制訊號
都在這條高速公路上跑，
讓各單元緊密合作！

Data
Moves
The World! 😊

Internal Bus
就像 CPU 裡的
高速公路，
負責在各單元之間
傳送資料、指令
和控制訊號，
讓整個 CPU
順暢運作！

拓真 Takuma

熱愛科技・勇於挑戰

Bus 的三種類型

資料匯流排 (Data Bus)	傳送資料 (例如 8 / 16 / 32 / 64 位元)
位址匯流排 (Address Bus)	傳送記憶體位址。 (決定位址範圍)
控制匯流排 (Control Bus)	傳送控制訊號 (例如 Read / Write / Interrupt)

大家透過
Internal Bus
緊密連結！
一起合作完成
任務！

Bus 越順暢，
CPU 的工作
就越有效率！

沒有 Bus，
各個單元就像
孤島一樣，
無法交換資料！
Internal Bus
讓 CPU 內部
的資料流動
又快又便利！



玲奈 Reina

喜歡思考・樂於學習

重點整理

- ✓ Internal Bus 是 CPU 內部各單元之間傳送資料、位址與控制訊號的通道。
- ✓ 資料匯流排 (Data Bus)：傳送資料。
- ✓ 位址匯流排 (Address Bus)：傳送記憶體位址。
- ✓ 控制匯流排 (Control Bus)：傳送控制訊號。
- ✓ Bus 讓 PC、暫存器、ALU、記憶體與 I/O 能緊密合作，是 CPU 正常運作不可或缺的基礎。

小結

- Internal Bus 就像 CPU 裡的高速公路，讓資料、位址與控制訊號在各單元之間快速傳送。
- 有了 Bus，CPU 的各個單元才能同步協作，發揮最大的效能，完成各種複雜的任務，讓我們的電腦運作更加順暢！

一條 Bus，
串起整個
CPU 的力量！

想一想...

- 1 為什麼 CPU 內部需要 Bus？如果沒有 Bus 會發生什麼情況？
- 2 資料匯流排、位址匯流排和控制匯流排有什麼不同？各自的用途是什麼？
- 3 你覺得如果 Bus 的傳輸速度變慢，會對 CPU 的運作造成什麼影響？



1-28

Intel 8085：從基本架構到實際微處理機

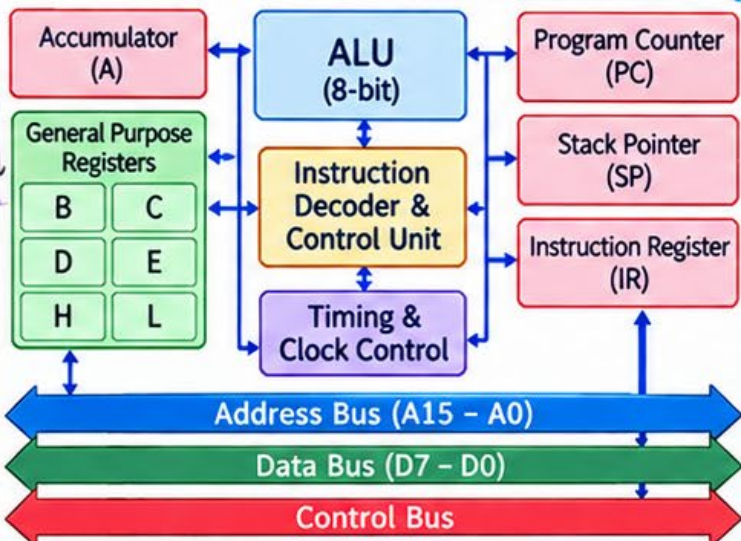
Intel 8085: From Basic Architecture to a Real Microprocessor

前面學到的 CPU 內部單元，
在 Intel 8085 中都有實際存在！
現在來看看真實的架構吧！

Theoretical
Concepts
→
Real Hardware

這就是 Intel 8085
的內部架構圖！
我們在前面的單元
(ALU、Register、
PC、Control Unit、
Internal Bus)
都能找到對應的
位置喔！

Intel 8085 內部架構圖

Intel 8085
8 位微處理機
(1976)

Intel 8085 主要規格

位元組 (Data Width)	8 位元 (8-bit)
位址總線 (Address Bus)	16 位元 (A15 - A0) (可尋址 64 KB)
資料總線 (Data Bus)	8 位元 (D7 - D0)
指令集 (Instruction Set)	約 74 條指令
晶片腳位 (Pins)	40 腳 (DIP)
時脈頻率 (Clock)	3 MHz (常見)
製造商	Intel

這顆小小的晶片
就是 8085！
雖然沒有華麗的外觀，
但它是許多早期
電腦與控制系統
的核心！

拓真 Takuma

熱愛科技・勇於挑戰

玲奈 Reina

喜歡思考・樂於學習



重點整理

- ✓ Intel 8085 是 8 位元微處理器，具備 16 位元位址總線與 8 位元資料總線。
- ✓ 內部包含 ALU、暫存器、指令解碼與控制單元、時序與時脈控制等關鍵單元。
- ✓ 可尋址 64 KB 記憶體，廣泛應用於早期電腦、工業控制與嵌入式系統。
- ✓ 其基本架構與我們前面學到的 CPU 內部單元完全對應。



小結

Intel 8085 的架構圖，
就是前面各個單元的「真實版」。
從這張圖中，我們可以清楚看到
CPU、ALU、暫存器、PC、SP、
控制單元與匯流排的實際位置
與連接方式。

理論×實際
才能真正
理解微處理機！



想一想...

- 1 Intel 8085 的位址總線與資料總線分別有幾位元？
- 2 除了 8085 之外，你還知道哪些早期的微處理機？
- 3 為什麼 8085 能夠廣泛應用於早期電腦與工業控制系統？



1-29

CPU、MCU、SoC 有什麼不同？

CPU vs. MCU vs. SoC : What's the Difference?

從單純運算，到完整系統，
不同的需求有不同的選擇，
讓我們找到最適合的方案！

Different
Chips
Different
Possibilities!

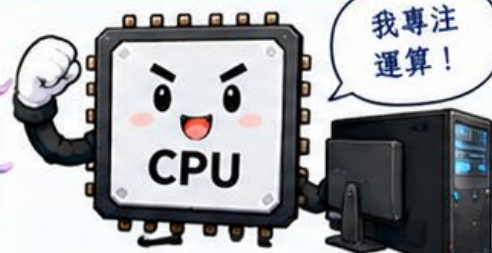
CPU、MCU、SoC
雖然都在運算，
但整合的功能、
應用場景與設計
目標都不同！
選對才能發揮
最大效益！

拓真 Takuma

熱愛科技，勇於挑戰

CPU

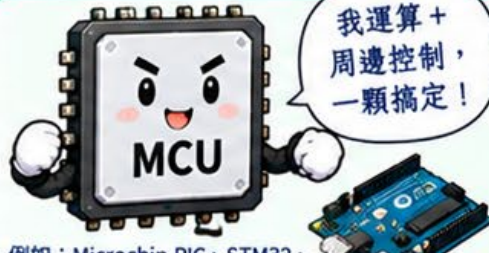
(中央處理器)



例如：Intel Core、AMD Ryzen

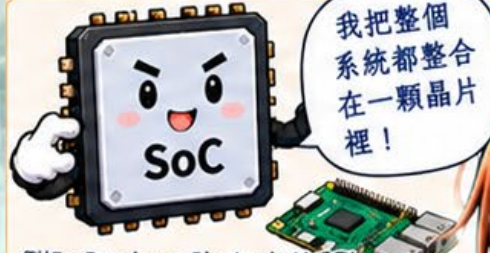
MCU

(微控制器)

例如：Microchip PIC、STM32、
Arduino (ATmega)

SoC

(系統單晶片)

例如：Raspberry Pi、Apple M 系列、
Qualcomm Snapdragon

整合程度	只有處理器核心	處理器核心 + 內建周邊	處理器 + 多種系統功能整合
主要功能	高速運算、執行指令	運算 + 控制周邊裝置	運算 + 圖形 + 多媒體 + 周邊 + I/O
周邊元件	需要外部記憶體與 I/O	內建 Flash、RAM、GPIO、 計時器、ADC...	內建 CPU、GPU、記憶體控制器、 影像、音訊、網路等
應用領域	個人電腦、伺服器、高效能運算	嵌入式控制、家電、工業控制、 物聯網裝置	智慧手機、平板、單板電腦、 多媒體裝置
代表例子	Intel Core、AMD Ryzen	Microchip PIC、STM32、 Arduino (ATmega)	Raspberry Pi、Apple M 系列、 Qualcomm Snapdragon

不像大小
決定強弱，
而是看需求！
CPU 適合高效
運算，MCU 適合
控制應用，SoC
則是把整個系統
整合在一起，
讓裝置更小、
更強大！

玲奈 Reina

喜歡思考，樂於學習



重點整理

- ✓ CPU、MCU、SoC 都是可以執行指令的處理器。
- ✓ CPU 主要負責高速運算，通常需要外部周邊元件。
- ✓ MCU 內建多種周邊功能，適合嵌入式控制應用。
- ✓ SoC 將 CPU、GPU、記憶體與多種功能整合在單一晶片。
- ✓ 選擇哪一種，取決於應用需求、效能、體積與成本的考量。



小結

CPU、MCU、SoC 並沒有絕對的好壞，
而是針對不同的應用場景而設計。
從個人電腦到智慧家電、從工業控制
到智慧手機，各種裝置都能找到最適
合的處理器型式。
了解它們的差異，
才能在未來的學習與應用中
做出正確的選擇！

不同的舞台，
不同的主角，
都是不可或缺的！

想一想...

- 1 你覺得日常生活中，
哪些裝置使用 CPU？哪些使用 MCU？
哪些使用 SoC？
- 2 如果要設計一個智慧溫控器，
你會選擇 CPU、MCU 還是 SoC？為什麼？
- 3 為什麼智慧手機大多使用 SoC？
這樣的設計有什麼優點？



1-30

Raspberry Pi 是 CPU 還是電腦？

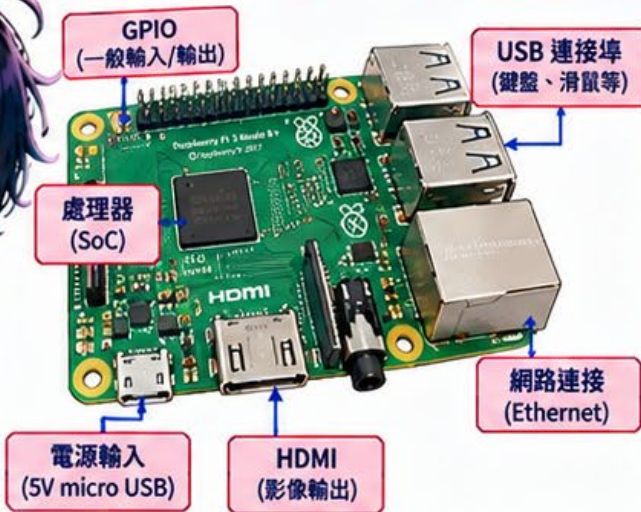
Raspberry Pi: A CPU or a Computer?

大家猜猜看！
Raspberry Pi
是 CPU、MCU、
SoC，還是一台完整的
電腦呢？

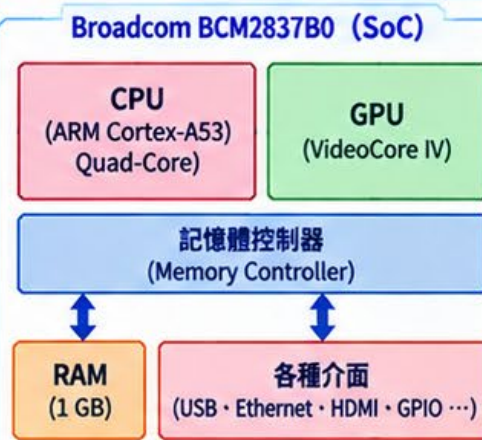
拓真 Takuma

熱愛科技・勇於挑戰

Raspberry Pi 3 Model B+

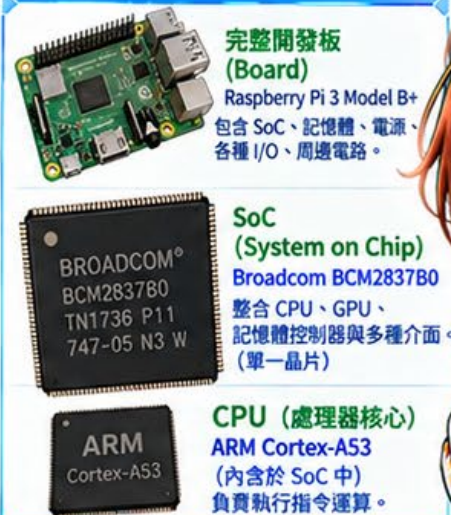


Raspberry Pi 的內部結構 (簡化圖)



一塊小小的板子，
卻能連上網路、跑程式、
控制裝置，Raspberry Pi
到底是什麼呢？

從板子到晶片的層級



Raspberry Pi
是一台「單板
電腦」(Single
Board Computer)
它使用 SoC，
但已經整合完成
可以直接使用
就像一台小型
電腦喔！

玲奈 Reina

喜歡思考・樂於學習



重點整理

- ✓ Raspberry Pi 3 Model B+ 採用 Broadcom BCM2837B0 SoC (內含 ARM Cortex-A53 CPU 與 VideoCore IV GPU)。
- ✓ Raspberry Pi 是一台單板電腦 (SBC)，已整合記憶體、儲存介面、網路、影像輸出與 GPIO 等功能。
- ✓ 它不是單純的 CPU，也不是傳統意義的 MCU，而是基於 SoC 的完整電腦系統。
- ✓ Raspberry Pi 適合用於教育、物聯網、控制、影像處理與各種創客應用。
- ✓ 不同應用需求，選擇 CPU・MCU・SoC 才能發揮最大效益。



小結：CPU・MCU・SoC・Raspberry Pi 比較

項目	CPU (處理器)	MCU (微控制器)	SoC (系統單晶片)	Raspberry Pi 3 B+ (單板電腦)
整合程度	僅處理器核心	CPU + 周邊	多核心 + 周邊	SoC + 記憶體 + 周邊電路 + 介面
主要用途	通用運算	嵌入式控制	行動裝置/嵌入式系統	教育、開發、多媒體應用
記憶體	外接	內建 (較小)	內建 (較大)	外接 (1 GB RAM)
作業系統	通常需要	通常不需要	可有可無	可安裝 (Linux 等)
代表例子	Intel / AMD	Arduino / STM32	手機 SoC	Raspberry Pi



想一想...

- 1 你認為 Raspberry Pi 是 CPU、MCU、SoC，還是完整的電腦？請說明你的理由。
- 2 為什麼 Raspberry Pi 適合應用在教育與物聯網領域？
- 3 如果只需要控制感測器與馬達，你會選擇 Raspberry Pi 還是 Arduino (MCU)？為什麼？



1-31

CPU 到底怎麼「跑程式」？

How Does a CPU Run a Program?

我們寫的程式，
會被轉換成
一連串指令。
CPU 就是依序
執行這些指令，
完成指定的工作！

拓真 Takuma

熱愛科技・勇於挑戰

我寫了一個簡單
的程式，想讓 CPU
把兩個數相加，
並把結果存到
記憶體中！
接下來就交給
CPU 吧！

玲奈 Reina

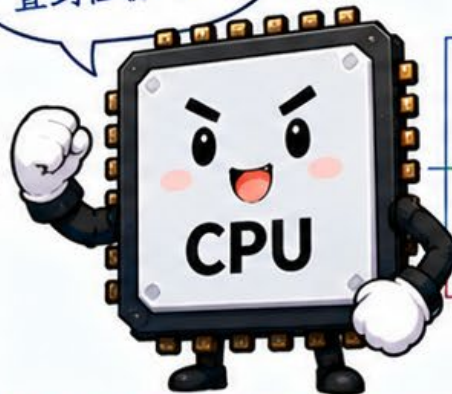
喜歡思考・樂於學習

Instruction Card

MOV A, 25H
ADD A, 10H
STA 2000H
...

指令序列
(機器碼)

沒問題！
我會一條一條
讀取、解碼、執行
你的指令，
直到任務完成！

Fetch
取指令Decode
解碼Execute
執行

程式 = 許多指令的組合
CPU 依序執行這些指令，
就能完成複雜的工作！

完成任務



計算



控制



資料處理



裝置控制

...



重點整理

- ✓ CPU 執行程式是依序處理一連串指令。
- ✓ 每一條指令都由「取指令 → 解碼 → 執行」完成。
- ✓ 程式通常由高階語言撰寫，再編譯成機器碼指令。
- ✓ CPU 重複執行指令，直到程式結束。
- ✓ 透過指令的組合，CPU 可以完成計算、控制、資料處理與各種應用。



小結

CPU 並不會一次看完整個程式，
而是從記憶體中依照程式計數器（PC）
所指示的位址，一條一條地取出指令，
經過解碼後執行，再繼續下一條指令。
透過這樣不斷重複的過程，
CPU 就能逐步完成我們設計的程式，
讓電腦執行各種任務。

一條指令，
改變世界！



想一想...

- 1 你覺得 CPU 是先把整個程式讀進來，還是一條一條執行？為什麼？
- 2 如果沒有 CPU，電腦能自己完成工作嗎？為什麼？
- 3 你在日常生活中，還看過哪些裝置也是依照指令一步一步執行任務？



Program
= Ideas
+ Instructions
+ CPU Works! 😊

一張一張指令，
CPU 依序執行，
就能完成我們想要的功能！

1-32

CPU 的指令週期：取指・解碼・執行

The Instruction Cycle: Fetch, Decode, Execute

看似簡單的三個步驟，
卻是電腦能完成各種工作的
關鍵循環！

Fetch
Decode
Execute
= Real Computation!

不管是 8085
還是現代的 CPU，
執行指令的流程
都是「取指、解碼、
執行」這個循環，
一條接一條，
不斷重複！

 μP

拓真 Takuma

熱愛科技・勇於挑戰

程式範例 (8085 組合語言)

```
2000H MVI A, 25H
2002H MVI B, 10H
2004H ADD B
2005H STA 3000H
2008H HLT
...
```

CPU 指令週期 (Instruction Cycle)

1 取指 (Fetch)

RAM
(記憶體)取出指令
(Instruction)
0011110 ...

- PC 指向下一條指令
- 從記憶體讀取指令
- PC 自動遞增
(指向下一個位址)

2 解碼 (Decode)

解譯指令
(Instruction
Decoding)

- 控制單元分析操作碼
- 判斷需要的資料與操作
- 產生控制信號
(控制各單元協同工作)

3 執行 (Execute)

暫存器
(Register)
記憶體
(Memory)

- 由 ALU、暫存器等單元
執行指令
- 完成資料運算、搬移或
輸出操作
- 更新旗標 (Flag) 與狀態

重複下一條指令 (Next Instruction)

CPU 就像一位
非常認真的學生，
不斷地「取、解、
執行」每一條指令，
如此就能完成
整個程式的工作！

玲奈 Reina

喜歡思考・樂於學習



重點整理

- ✓ CPU 執行指令的流程為：取指 (Fetch) → 解碼 (Decode) → 執行 (Execute)。
- ✓ 程式通常存放在記憶體中，CPU 依序讀取並執行。
- ✓ PC (程式計數器) 記錄下一條指令的位址，每次取指後會自動遞增。
- ✓ 控制單元負責解碼並產生控制信號，協調各單元完成操作。
- ✓ 這個循環會一條指令一條指令地不斷重複，直到遇到結束指令 (例如 HLT)。



小結

CPU 的強大來自於不斷重複的「取指、解碼、執行」循環。雖然每一步看起來很簡單，但透過高速的時脈與精密的控制，CPU 能在極短時間內完成大量指令，讓電腦具備強大的運算與控制能力。理解這個流程，是進一步學習組合語言與系統程式設計的基礎！

簡單的循環，
創造無限的
可能！

想一想...

- 1 如果 PC 沒有自動遞增，會發生什麼情況？
- 2 為什麼需要「解碼」這個步驟？不能直接執行嗎？
- 3 現代的 CPU 在一個時脈週期內只能執行一條指令嗎？有沒有更快的方法？



1-33

1 Fetch：把下一個指令拿進來

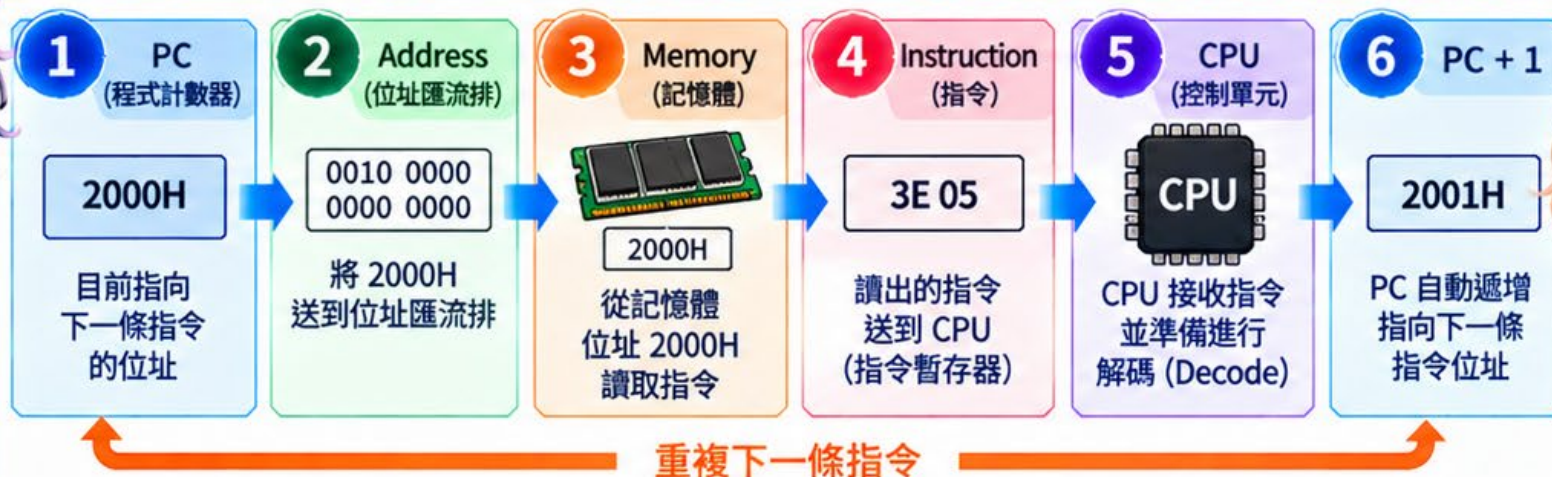
Fetch: Retrieve the Next Instruction

程式執行的第一步，
就是先把下一個指令
從記憶體拿進 CPU！

Fetch
decode
execute
Repeat！

CPU 不是一次
把整個程式
讀進來，而是
依照 PC 指示的
位址，一次讀取
一條指令！

指令擷取 (Fetch) 流程



Fetch 的重點
就是「依照 PC
讀出一條指令」，
讀完後 PC 會自動
加 1，準備下一次
的取指！

拓真 Takuma

熱愛科技・勇於挑戰

玲奈 Reina

喜歡思考・樂於學習



重點整理

- ✓ CPU 依照 PC 內容指示的位址，從記憶體讀取一條指令 (Fetch)。
- ✓ 讀出的指令會被送到 CPU 的指令暫存器。
- ✓ 取指完成後，PC 會自動遞增，指向下一條指令的位址。
- ✓ CPU 並不會一次讀進整個程式，而是每次取一條指令，依序執行。
- ✓ Fetch 是指令週期的第一步，接著才進行解碼 (Decode) 與執行 (Execute)。



小結

Fetch 就是程式執行的起點。
CPU 依 PC 指示的位址，
從記憶體讀取下一條指令，
並在取指完成後將 PC 自動加 1。
這個簡單的動作會不斷重複，
讓 CPU 能依序執行整個程式。

一條指令，
開啟無限可能！

想一想...

- 1 為什麼需要 PC 來記錄「下一條指令的位址」？
- 2 如果取指後沒有自動加 1，會發生什麼情況？
- 3 CPU 為什麼不一次把整個程式讀進來，而是每次取一條指令？



1-34

2 Decode：這個指令要我做什麼？

Decode: What Does This Instruction Mean?

CPU 需要看懂指令，
才能知道要做什麼工作！
Decode 就像翻譯員，
把機器碼變成能理解的意思！

Decode
= Understand
+ Plan!

電腦不會魔法，
它必須先看懂
這個指令代表
什麼意思，
才知道後續
要做什麼！

拓真 Takuma

熱愛科技・勇於挑戰

指令解碼 (Decode) 的工作

機器碼指令
(Instruction)

3E 05

讓我來
翻譯這條指令
的意思！

Instruction
Decoder
(指令解碼器)

拆解成兩個部分

運算子
(Operator)

3E

代表動作
(例如：MVI)

運算元
(Operand)

05

代表資料
(例如：5)

8085 指令範例

機器碼 (Hex)	運算子 (Operator)	運算元 (Operand)	指令意義 (組合語言)
3E 05	3E	05	MVI A, 05H
06 0A	06	0A	MVI B, 0AH
80 --	80	--	ADD B (A = A + B)
32 00 20	32	00 20	STA 2000H (儲存 A 到 2000H)

Decode 的任務
就是把機器碼
拆成「運算子」
和「運算元」，
並產生控制信號，
讓後續的執行單元
知道要做什麼！

玲奈 Reina

喜歡思考・樂於學習



重點整理

- ✓ Decode 負責把取來的機器碼指令進行解析。
- ✓ 一條指令通常由「運算子 (Operator)」和「運算元 (Operand)」組成。
- ✓ 運算子決定要做什麼動作 (如 MVI、ADD、STA 等)。
- ✓ 運算元提供動作所需的資料 (如 05H、0AH、位址等)。
- ✓ 指令解碼器會產生控制信號，送到各個單元 (暫存器、ALU、記憶體等)。
- ✓ 這個過程就像翻譯員，把機器碼翻譯成 CPU 能理解的控制指令。



小結

Decode 是 CPU 的「理解」階段，
它把機器碼指令拆解成
運算子與運算元，
並產生對應的控制信號，
讓後續的執行單元知道
要完成什麼工作。
只有看懂指令，CPU 才能正確執行！

看懂指令，
才能正確行動！



想一想...

- 1 如果把 3E 05 換成 06 0A，
Decode 後會得到什麼指令？
- 2 為什麼需要把指令拆成
「運算子」和「運算元」？
- 3 如果指令中沒有運算元 (例如 80)
Decode 需要做的工作有什麼不同？



1-35

3

Execute + Pipeline：執行與指令流水線

Execute + Pipeline: ALU Execution and Instruction Pipeline

一條指令的完成，
不只是執行，
更可以透過流水線設計，
讓多條指令同時進行！

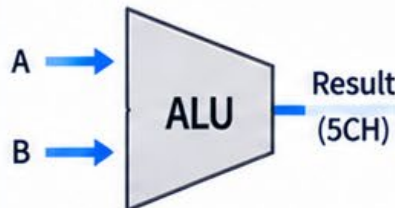
Fetch
Decode
Execute
= Faster CPU!

解碼之後，
CPU 就知道要做
什麼了，接下來
就交給 ALU 與
各單元執行，
完成運算或
資料處理！

ALU 的執行 (Execute)

加法運算範例

A = 25H
B = 37H
Result = 5CH



可能的運算

- 加法 (ADD)
- 減法 (SUB)
- 邏輯運算 (AND, OR, XOR)
- 比較 (CMP)
- 移位 (SHIFT)
- 其他運算 ...

執行後的結果

結果 5CH
(0101 1100)

結果會寫回暫存器
或記憶體中

Flags 狀態	
Z	0
S	0
C	0
P	1

透過 ALU 執行
運算、邏輯或比較，
就能得到結果！
而指令流水線
可以讓 CPU 同時
處理多條指令，
大幅提升
執行效率！

指令流水線 (Instruction Pipeline)

多條指令的三個階段同時進行，提升 CPU 的執行效率！

指令序列

I1
I2
I3
I4
...

Fetch 取指

取出
下一條指令

Decode 解碼

分析
指令內容

Execute 執行

執行運算
並寫回結果

流水線讓 CPU
像工廠生產線一樣，
多條指令同時進行，
提升整體效能！

拓真 Takuma

熱愛科技・勇於挑戰

玲奈 Reina

喜歡思考・樂於學理



重點整理

- ✓ Execute 階段由 ALU 及各功能單元執行指令。
- ✓ 運算結果會寫回暫存器或記憶體。
- ✓ 常見的運算包含：加法、減法、邏輯運算、比較、移位等。
- ✓ 指令流水線 (Pipeline) 將指令分為 Fetch、Decode、Execute 三個階段。
- ✓ 流水線讓多條指令同時進行，提升 CPU 的執行效率。
- ✓ 現代 CPU 通常具有更多階段的流水線，以達到更高的效能。

小結

Execute 是指令執行的核心，
而指令流水線則是提升效能的關鍵技術。
透過多條指令的重疊處理，
CPU 可以在同一時間完成多項工作，
就像工廠的生產線一樣，
讓電腦運算更快、更有效率！

同時處理多條指令，
讓 CPU 更強大！

想一想...

- 1 如果沒有 ALU，CPU 還能做什麼工作？
- 2 為什麼需要指令流水線 (Pipeline)？它帶來什麼好處？
- 3 如果某一條指令需要較長的執行時間，會對流水線產生什麼影響？有什麼方法可以改善？



1-36

Chapter 1 Mission Complete!

從概念到執行，你已經完成第一章的學習！

Great Job! You've completed Chapter 1!

打好基礎，
下一載我們將進入
更有趣的應用！Small
Concepts
Big Possibilities!
Keep Going!

全章概念地圖 (Chapter 1 Concept Map)

我們
是一個團隊！運算
我來負責！資料
交給我！告訴我
下一步！我來
指揮協調！我讓大家
連在一起！Team μP !

拓真 Takuma

熱愛科技・勇於挑戰

3 題快速 Checkpoint

1

下列哪一個單元負責進行算術與邏輯運算？

- (A) 控制單元 (B) 暫存器 (C) ALU (D) 匯流排

2

在指令週期中，CPU 首先會做什麼？

- (A) 執行指令 (B) 解碼指令 (C) 取指令 (D) 將資料寫入記憶體

3

在 Intel 8085 中，指令「3E 05」的運算子 (Operator) 與運算元 (Operand) 分別是什麼？

- (A) 3E, 05 (B) 05, 3E (C) 3E, 00 (D) 05, 00

想想看，
你答對幾題？

給未來的你

你已經完成了第一章的學習，
這些基礎將是你未來設計、
分析與應用微處理機的重要基石。
保持好奇，持續練習，
你一定能走得更遠！

基礎扎實，
才能創造無限可能！

5 個關鍵重點 (Key Takeaways)

1

微處理機的角色

微處理機是電腦系統的核心，負責資料處理與控制。

2

數字系統是基礎

二進位、十六進位、補數等觀念是理解指令與資料的基礎。

3

硬體組成與功能

ALU、暫存器、PC、控制單元、匯流排各有其重要功能。

4

指令週期的流程

Fetch → Decode → Execute 是 CPU 執行指令的基本流程。

5

Intel 8085 的實例

透過 8085，我們將抽象概念連結到實際的微處理機。

第一章
完成了！
接下來
我們繼續挑戰
更進階的內容
吧！Good
Job!

玲奈 Reina

喜歡思考・樂於學習

Next Chapter
More Fun!

Level Up!